
◆ Basic Number Problems in TypeScript

1. Check if a number is even or odd

```
// ✔ Using inbuilt
function isEvenBuiltIn(n: number): boolean {
    return n % 2 === 0;
}

// ☒ Without using inbuilt (%)
function isEvenNoMod(n: number): boolean {
    while (n >= 2) n -= 2;
    return n === 0;
}

console.log(isEvenBuiltIn(10)); // true
console.log(isEvenNoMod(7));   // false
```

2. Find the largest of three numbers

```
// ✔ Using inbuilt
function largestBuiltIn(a: number, b: number, c: number): number {
    return Math.max(a, b, c);
}

// ☒ Without inbuilt
function largestNoBuiltIn(a: number, b: number, c: number): number {
    let max = a;
    if (b > max) max = b;
    if (c > max) max = c;
    return max;
}
```

3. Swap two numbers without using third variable

```
// ✔ Using array destructuring
let x = 5, y = 10;
[x, y] = [y, x];
console.log(x, y);

// ☒ Without inbuilt (arithmetic)
let a = 5, b = 10;
a = a + b;
b = a - b;
a = a - b;
console.log(a, b);
```

4. Find sum of digits of a number

```
// ✔ Using inbuilt
function sumDigitsBuiltIn(n: number): number {
  return n.toString().split('').reduce((sum, d) => sum + parseInt(d), 0);
}

// ☒ Without inbuilt
function sumDigitsNoBuiltIn(n: number): number {
  let sum = 0;
  while (n > 0) {
    sum += n % 10;
    n = Math.floor(n / 10);
  }
  return sum;
}
```

5. Reverse a number

```
// ✔ Using inbuilt
function reverseNumberBuiltIn(n: number): number {
  return parseInt(n.toString().split('').reverse().join(''));
}

// ☒ Without inbuilt
function reverseNumberNoBuiltIn(n: number): number {
  let rev = 0;
  while (n > 0) {
    rev = rev * 10 + (n % 10);
    n = Math.floor(n / 10);
  }
  return rev;
}
```

6. Check if a number is a palindrome

```
// ✔ Using inbuilt
function isPalindromeBuiltIn(n: number): boolean {
  let str = n.toString();
  return str === str.split('').reverse().join('');
}

// ☒ Without inbuilt
function isPalindromeNoBuiltIn(n: number): boolean {
  let original = n, rev = 0;
  while (n > 0) {
    rev = rev * 10 + (n % 10);
    n = Math.floor(n / 10);
  }
  return rev === original;
}
```

7. Factorial of a number

```
// ✔ Using recursion (inbuilt style)
function factorialBuiltIn(n: number): number {
  if (n <= 1) return 1;
  return n * factorialBuiltIn(n - 1);
}

// ⓪ Without recursion (loop)
function factorialNoBuiltIn(n: number): number {
  let fact = 1;
  for (let i = 2; i <= n; i++) fact *= i;
  return fact;
}
```

8. Count digits in a number

```
// ✔ Using inbuilt
function countDigitsBuiltIn(n: number): number {
  return n.toString().length;
}

// ⓪ Without inbuilt
function countDigitsNoBuiltIn(n: number): number {
  let count = 0;
  while (n > 0) {
    count++;
    n = Math.floor(n / 10);
  }
  return count;
}
```

9. Find GCD (HCF) of two numbers

```
// ✔ Using inbuilt Euclidean algo
function gcdBuiltIn(a: number, b: number): number {
  return b === 0 ? a : gcdBuiltIn(b, a % b);
}

// ⓪ Without inbuilt (loop)
function gcdNoBuiltIn(a: number, b: number): number {
  while (a !== b) {
    if (a > b) a -= b;
    else b -= a;
  }
  return a;
}
```

10. Find LCM of two numbers

```
// ✔ Using inbuilt Math + GCD
```

```
function lcmBuiltIn(a: number, b: number): number {
  return (a * b) / gcdBuiltIn(a, b);
}

// ⓪ Without inbuilt
function lcmNoBuiltIn(a: number, b: number): number {
  let max = a > b ? a : b;
  while (true) {
    if (max % a === 0 && max % b === 0) return max;
    max++;
  }
}
```

◆ Prime Numbers (11–20)

11. Check if a number is prime

```
// ✓ Using inbuilt sqrt
function isPrimeBuiltIn(n: number): boolean {
  if (n < 2) return false;
  for (let i = 2; i <= Math.sqrt(n); i++) {
    if (n % i === 0) return false;
  }
  return true;
}

// ⓧ Without inbuilt sqrt
function isPrimeNoBuiltIn(n: number): boolean {
  if (n < 2) return false;
  let i = 2;
  while (i * i <= n) {
    if (n % i === 0) return false;
    i++;
  }
  return true;
}
```

12. Print all prime numbers up to n

```
// ✓ Using inbuilt sqrt
function primesUpToBuiltIn(n: number): number[] {
  const primes: number[] = [];
  for (let i = 2; i <= n; i++) {
    if (isPrimeBuiltIn(i)) primes.push(i);
  }
  return primes;
}

// ⓧ Without inbuilt sqrt
function primesUpToNoBuiltIn(n: number): number[] {
  const primes: number[] = [];
  for (let i = 2; i <= n; i++) {
    if (isPrimeNoBuiltIn(i)) primes.push(i);
  }
  return primes;
}
```

13. Find the n th prime number

```
// ✓ Using helper prime check
```

```

function nthPrimeBuiltIn(n: number): number {
  let count = 0, num = 1;
  while (count < n) {
    num++;
    if (isPrimeBuiltIn(num)) count++;
  }
  return num;
}

// ⓪ Without inbuilt
function nthPrimeNoBuiltIn(n: number): number {
  let count = 0, num = 1;
  while (count < n) {
    num++;
    if (isPrimeNoBuiltIn(num)) count++;
  }
  return num;
}

```

14. Prime factorization of a number

```

// ✔ Using sqrt
function primeFactorsBuiltIn(n: number): number[] {
  const factors: number[] = [];
  while (n % 2 === 0) {
    factors.push(2);
    n /= 2;
  }
  for (let i = 3; i <= Math.sqrt(n); i += 2) {
    while (n % i === 0) {
      factors.push(i);
      n /= i;
    }
  }
  if (n > 2) factors.push(n);
  return factors;
}

// ⓪ Without sqrt
function primeFactorsNoBuiltIn(n: number): number[] {
  const factors: number[] = [];
  let i = 2;
  while (i * i <= n) {
    while (n % i === 0) {
      factors.push(i);
      n /= i;
    }
    i++;
  }
  if (n > 1) factors.push(n);
  return factors;
}

```

15. Find the largest prime factor

```
// ✔ Using inbuilt
function largestPrimeFactorBuiltIn(n: number): number {
  const factors = primeFactorsBuiltIn(n);
  return Math.max(...factors);
}

// ⓧ Without inbuilt
function largestPrimeFactorNoBuiltIn(n: number): number {
  let largest = -1, i = 2;
  while (i * i <= n) {
    while (n % i === 0) {
      largest = i;
      n /= i;
    }
    i++;
  }
  if (n > 1) largest = n;
  return largest;
}
```

16. Count total prime factors of a number

```
// ✔ Using inbuilt
function countPrimeFactorsBuiltIn(n: number): number {
  return primeFactorsBuiltIn(n).length;
}

// ⓧ Without inbuilt
function countPrimeFactorsNoBuiltIn(n: number): number {
  let count = 0, i = 2;
  while (i * i <= n) {
    while (n % i === 0) {
      count++;
      n /= i;
    }
    i++;
  }
  if (n > 1) count++;
  return count;
}
```

17. Check if a number is a twin prime

☞ Twin primes are primes that differ by 2 (e.g., 11 and 13).

```
// ✔ Using inbuilt
function isTwinPrimeBuiltIn(n: number): boolean {
  if (!isPrimeBuiltIn(n)) return false;
  return isPrimeBuiltIn(n - 2) || isPrimeBuiltIn(n + 2);
}

// ⓧ Without inbuilt
function isTwinPrimeNoBuiltIn(n: number): boolean {
  if (!isPrimeNoBuiltIn(n)) return false;
  if (!isPrimeNoBuiltIn(n - 2)) return false;
  if (!isPrimeNoBuiltIn(n + 2)) return false;
  return true;
}
```

```
    return isPrimeNoBuiltIn(n - 2) || isPrimeNoBuiltIn(n + 2);
}
```

18. Check if a number is a Sophie Germain prime

☞ A **Sophie Germain prime** is a prime p such that $2p + 1$ is also prime.

```
// ✔ Using inbuilt
function isSophieGermainBuiltIn(n: number): boolean {
    return isPrimeBuiltIn(n) && isPrimeBuiltIn(2 * n + 1);
}

// ☒ Without inbuilt
function isSophieGermainNoBuiltIn(n: number): boolean {
    return isPrimeNoBuiltIn(n) && isPrimeNoBuiltIn(2 * n + 1);
}
```

19. Check if two numbers are co-prime

☞ Two numbers are co-prime if $\text{GCD} = 1$.

```
// ✔ Using gcdBuiltIn
function areCoPrimeBuiltIn(a: number, b: number): boolean {
    return gcdBuiltIn(a, b) === 1;
}

// ☒ Without inbuilt
function areCoPrimeNoBuiltIn(a: number, b: number): boolean {
    return gcdNoBuiltIn(a, b) === 1;
}
```

20. Implement Sieve of Eratosthenes

```
// ✔ Using arrays
function sieveBuiltIn(n: number): number[] {
    const prime: boolean[] = new Array(n + 1).fill(true);
    prime[0] = prime[1] = false;
    for (let i = 2; i <= Math.sqrt(n); i++) {
        if (prime[i]) {
            for (let j = i * i; j <= n; j += i) prime[j] = false;
        }
    }
    return prime.map((isP, i) => (isP ? i : -1)).filter(x => x !== -1);
}

// ☒ Without Math.sqrt
function sieveNoBuiltIn(n: number): number[] {
    const prime: boolean[] = new Array(n + 1).fill(true);
    prime[0] = prime[1] = false;
    let i = 2;
    while (i * i <= n) {

```



```
    if (prime[i]) {
      let j = i * i;
      while (j <= n) {
        prime[j] = false;
        j += i;
      }
    }
    i++;
  }
  const result: number[] = [];
  for (let k = 2; k <= n; k++) if (prime[k]) result.push(k);
  return result;
}
```

◆ Special Numbers (21–30)

21. Armstrong Number

☞ A number is Armstrong if the sum of its digits each raised to power = number of digits equals the number itself.

```
// ✔ Using inbuilt
function isArmstrongBuiltIn(n: number): boolean {
  const digits = n.toString().split('').map(Number);
  const power = digits.length;
  const sum = digits.reduce((acc, d) => acc + Math.pow(d, power), 0);
  return sum === n;
}

// ☒ Without inbuilt
function isArmstrongNoBuiltIn(n: number): boolean {
  let temp = n, digits = 0, sum = 0;
  let t = n;
  while (t > 0) { digits++; t = Math.floor(t / 10); }
  t = n;
  while (t > 0) {
    let digit = t % 10, p = 1;
    for (let i = 0; i < digits; i++) p *= digit;
    sum += p;
    t = Math.floor(t / 10);
  }
  return sum === n;
}
```

22. Strong Number

☞ Sum of factorial of digits = number. Example: 145.

```
// ✔ Using factorial helper
function isStrongBuiltIn(n: number): boolean {
  const factorial = (x: number) => x <= 1 ? 1 : x * factorial(x - 1);
  const sum = n.toString().split('').map(Number).reduce((acc, d) => acc + factorial(d), 0);
  return sum === n;
}

// ☒ Without inbuilt
function isStrongNoBuiltIn(n: number): boolean {
  let temp = n, sum = 0;
  while (temp > 0) {
    let digit = temp % 10;
    let fact = 1;
    for (let i = 2; i <= digit; i++) fact *= i;
    sum += fact;
    temp = Math.floor(temp / 10);
  }
}
```

```
    }  
    return sum === n;  
}
```

23. Perfect Number

☞ Sum of divisors (excluding itself) = number. Example: 28.

```
// ✔ Using inbuilt  
function isPerfectBuiltIn(n: number): boolean {  
    let sum = 0;  
    for (let i = 1; i <= n / 2; i++) if (n % i === 0) sum += i;  
    return sum === n;  
}  
  
// ⓧ Without inbuilt (manual divisor check)  
function isPerfectNoBuiltIn(n: number): boolean {  
    let sum = 0, i = 1;  
    while (i <= n / 2) {  
        if (n % i === 0) sum += i;  
        i++;  
    }  
    return sum === n;  
}
```

24. Harshad (Niven) Number

☞ Divisible by sum of digits.

```
// ✔ Using inbuilt  
function isHarshadBuiltIn(n: number): boolean {  
    const sum = n.toString().split('').map(Number).reduce((a, d) => a + d, 0);  
    return n % sum === 0;  
}  
  
// ⓧ Without inbuilt  
function isHarshadNoBuiltIn(n: number): boolean {  
    let temp = n, sum = 0;  
    while (temp > 0) {  
        sum += temp % 10;  
        temp = Math.floor(temp / 10);  
    }  
    return n % sum === 0;  
}
```

25. Neon Number

☞ Sum of digits of square of number = number. Example: $9 \rightarrow 81 \rightarrow 8+1=9$.

```
// ✔ Using inbuilt
```

```
function isNeonBuiltIn(n: number): boolean {
  const sq = n * n;
  const sum = sq.toString().split('').map(Number).reduce((a, d) => a + d, 0);
  return sum === n;
}
```

```
// ⓪ Without inbuilt
function isNeonNoBuiltIn(n: number): boolean {
  let sq = n * n, sum = 0;
  while (sq > 0) {
    sum += sq % 10;
    sq = Math.floor(sq / 10);
  }
  return sum === n;
}
```

26. Spy Number

☞ Sum of digits = Product of digits. Example: 1124 $\rightarrow$ (1+1+2+4=8, 112*4=8).

```
// ✔ Using inbuilt
function isSpyBuiltIn(n: number): boolean {
  const digits = n.toString().split('').map(Number);
  const sum = digits.reduce((a, d) => a + d, 0);
  const product = digits.reduce((a, d) => a * d, 1);
  return sum === product;
}
```

```
// ⓪ Without inbuilt
function isSpyNoBuiltIn(n: number): boolean {
  let sum = 0, product = 1, temp = n;
  while (temp > 0) {
    let d = temp % 10;
    sum += d;
    product *= d;
    temp = Math.floor(temp / 10);
  }
  return sum === product;
}
```

27. Duck Number

☞ Contains zero but does not start with zero. Example: 3210.

```
// ✔ Using inbuilt
function isDuckBuiltIn(n: number): boolean {
  const str = n.toString();
  return str.includes("0") && str[0] !== "0";
}
```

```
// ⓪ Without inbuilt
function isDuckNoBuiltIn(n: number): boolean {
  let hasZero = false, firstDigit = -1, temp = n;
```

```

while (temp > 0) {
  let d = temp % 10;
  if (temp < 10) firstDigit = d;
  if (d === 0) hasZero = true;
  temp = Math.floor(temp / 10);
}
return hasZero && firstDigit !== 0;
}

```

28. Automorphic Number

☞ Square ends with the number itself. Example: $25^2=625 \rightarrow$ ends with 25.

```

// ✔ Using inbuilt
function isAutomorphicBuiltIn(n: number): boolean {
  return (n * n).toString().endsWith(n.toString());
}

// ☒ Without inbuilt
function isAutomorphicNoBuiltIn(n: number): boolean {
  let sq = n * n, temp = n;
  while (temp > 0) {
    if (temp % 10 !== sq % 10) return false;
    temp = Math.floor(temp / 10);
    sq = Math.floor(sq / 10);
  }
  return true;
}

```

29. Magic Number

☞ Repeatedly sum digits until single digit $\rightarrow$ if result = 1, it's magic.

```

// ✔ Using inbuilt
function isMagicBuiltIn(n: number): boolean {
  while (n > 9) {
    n = n.toString().split('').map(Number).reduce((a, d) => a + d, 0);
  }
  return n === 1;
}

// ☒ Without inbuilt
function isMagicNoBuiltIn(n: number): boolean {
  while (n > 9) {
    let sum = 0, temp = n;
    while (temp > 0) {
      sum += temp % 10;
      temp = Math.floor(temp / 10);
    }
    n = sum;
  }
  return n === 1;
}

```

30. Happy Number

☞ Replace number by sum of squares of digits until reaching 1 (happy) or a cycle.

```
// ✔ Using inbuilt
function isHappyBuiltIn(n: number): boolean {
  const seen = new Set<number>();
  while (n !== 1 && !seen.has(n)) {
    seen.add(n);
    n = n.toString().split('').map(Number).reduce((a, d) => a + d * d, 0);
  }
  return n === 1;
}
```

```
// ☹ Without inbuilt
function isHappyNoBuiltIn(n: number): boolean {
  const seen: number[] = [];
  while (n !== 1 && !seen.includes(n)) {
    seen.push(n);
    let sum = 0, temp = n;
    while (temp > 0) {
      let d = temp % 10;
      sum += d * d;
      temp = Math.floor(temp / 10);
    }
    n = sum;
  }
  return n === 1;
}
```

◆ Fibonacci & Series (31–40)

31. Print Fibonacci series up to n

```
// ✔ Using array
function fibonacciSeriesBuiltIn(n: number): number[] {
  const series: number[] = [0, 1];
  for (let i = 2; i < n; i++) {
    series.push(series[i - 1] + series[i - 2]);
  }
  return series.slice(0, n);
}

// ⓪ Without array push
function fibonacciSeriesNoBuiltIn(n: number): void {
  let a = 0, b = 1;
  console.log(a);
  if (n > 1) console.log(b);
  for (let i = 2; i < n; i++) {
    let next = a + b;
    console.log(next);
    a = b;
    b = next;
  }
}
```

32. Find n th Fibonacci number

```
// ✔ Using recursion
function fibonacciNthBuiltIn(n: number): number {
  if (n <= 1) return n;
  return fibonacciNthBuiltIn(n - 1) + fibonacciNthBuiltIn(n - 2);
}

// ⓪ Using loop (manual)
function fibonacciNthNoBuiltIn(n: number): number {
  if (n <= 1) return n;
  let a = 0, b = 1, result = 0;
  for (let i = 2; i <= n; i++) {
    result = a + b;
    a = b;
    b = result;
  }
  return result;
}
```

33. Check if a number is in Fibonacci sequence

```
// ✔ Using math property ( $5 \cdot n^2 \pm 4$  must be perfect square)
```

```

function isPerfectSquare(x: number): boolean {
  const s = Math.floor(Math.sqrt(x));
  return s * s === x;
}

function isFibonacciBuiltIn(n: number): boolean {
  return isPerfectSquare(5 * n * n + 4) || isPerfectSquare(5 * n * n - 4);
}

// ⓪ Without sqrt (generate sequence)
function isFibonacciNoBuiltIn(n: number): boolean {
  let a = 0, b = 1;
  while (b < n) {
    let next = a + b;
    a = b;
    b = next;
  }
  return b === n || n === 0;
}

```

34. Sum of first n natural numbers

```

// ✔ Using formula
function sumNaturalBuiltIn(n: number): number {
  return (n * (n + 1)) / 2;
}

// ⓪ Without formula
function sumNaturalNoBuiltIn(n: number): number {
  let sum = 0;
  for (let i = 1; i <= n; i++) sum += i;
  return sum;
}

```

35. Sum of first n even numbers

```

// ✔ Using formula (n * (n+1))
function sumEvenBuiltIn(n: number): number {
  return n * (n + 1);
}

// ⓪ Without formula
function sumEvenNoBuiltIn(n: number): number {
  let sum = 0, count = 0, num = 2;
  while (count < n) {
    sum += num;
    num += 2;
    count++;
  }
  return sum;
}

```

36. Sum of first n odd numbers


```
// ✔ Using formula ( $n^2$ )
function sumOddBuiltIn(n: number): number {
  return n * n;
}

// ⓪ Without formula
function sumOddNoBuiltIn(n: number): number {
  let sum = 0, count = 0, num = 1;
  while (count < n) {
    sum += num;
    num += 2;
    count++;
  }
  return sum;
}
```

37. Find nth Triangular Number

☞ Formula = $n(n+1)/2$

```
// ✔ Using formula
function triangularNumberBuiltIn(n: number): number {
  return (n * (n + 1)) / 2;
}

// ⓪ Without formula
function triangularNumberNoBuiltIn(n: number): number {
  let sum = 0;
  for (let i = 1; i <= n; i++) sum += i;
  return sum;
}
```

38. Find nth Catalan Number

☞ Formula: $C_n = (2n)! / ((n+1)! * n!)$

```
// ✔ Using factorial helper
function factorial(n: number): number {
  return n <= 1 ? 1 : n * factorial(n - 1);
}
function catalanBuiltIn(n: number): number {
  return factorial(2 * n) / (factorial(n + 1) * factorial(n));
}

// ⓪ Without recursion
function catalanNoBuiltIn(n: number): number {
  let fact = (x: number): number => {
    let res = 1;
    for (let i = 2; i <= x; i++) res *= i;
    return res;
  };
  return fact(2 * n) / (fact(n + 1) * fact(n));
}
```

39. Pascal's Triangle (first n rows)

```
// ✓ Using array
function pascalTriangleBuiltIn(n: number): number[][] {
  const triangle: number[][] = [];
  for (let i = 0; i < n; i++) {
    triangle[i] = new Array(i + 1).fill(1);
    for (let j = 1; j < i; j++) {
      triangle[i][j] = triangle[i - 1][j - 1] + triangle[i - 1][j];
    }
  }
  return triangle;
}

// ⓧ Without new Array
function pascalTriangleNoBuiltIn(n: number): number[][] {
  const triangle: number[][] = [];
  for (let i = 0; i < n; i++) {
    triangle[i] = [];
    for (let j = 0; j <= i; j++) {
      if (j === 0 || j === i) triangle[i][j] = 1;
      else triangle[i][j] = triangle[i - 1][j - 1] + triangle[i - 1][j];
    }
  }
  return triangle;
}
```

40. Count number of ways to climb stairs

☞ Classic DP (like Fibonacci).

```
// ✓ Using DP array
function climbStairsBuiltIn(n: number): number {
  if (n <= 2) return n;
  const dp: number[] = [0, 1, 2];
  for (let i = 3; i <= n; i++) {
    dp[i] = dp[i - 1] + dp[i - 2];
  }
  return dp[n];
}

// ⓧ Without array (just variables)
function climbStairsNoBuiltIn(n: number): number {
  if (n <= 2) return n;
  let a = 1, b = 2, ways = 0;
  for (let i = 3; i <= n; i++) {
    ways = a + b;
    a = b;
    b = ways;
  }
  return ways;
}
```

◆ 41. Convert Decimal to Binary

✓ Using Built-in:

```
function decimalToBinaryBuiltIn(n: number): string {  
    return n.toString(2);  
}  
  
console.log(decimalToBinaryBuiltIn(10)); // "1010"
```

✓ Without Built-in:

```
function decimalToBinaryManual(n: number): string {  
    let binary = "";  
    while (n > 0) {  
        binary = (n % 2) + binary;  
        n = Math.floor(n / 2);  
    }  
    return binary || "0";  
}  
  
console.log(decimalToBinaryManual(10)); // "1010"
```

◆ 42. Convert Binary to Decimal

✓ Using Built-in:

```
function binaryToDecimalBuiltIn(bin: string): number {  
    return parseInt(bin, 2);  
}  
  
console.log(binaryToDecimalBuiltIn("1010")); // 10
```

✓ Without Built-in:

```
function binaryToDecimalManual(bin: string): number {  
    let result = 0;  
    for (let i = 0; i < bin.length; i++) {  
        result = result * 2 + (bin[i] === "1" ? 1 : 0);  
    }  
    return result;  
}  
  
console.log(binaryToDecimalManual("1010")); // 10
```

◆ 43. Convert Decimal to Octal

✓ Built-in:

```
function decimalToOctalBuiltIn(n: number): string {
    return n.toString(8);
}

console.log(decimalToOctalBuiltIn(64)); // "100"
```

✓ Manual:

```
function decimalToOctalManual(n: number): string {
    let octal = "";
    while (n > 0) {
        octal = (n % 8) + octal;
        n = Math.floor(n / 8);
    }
    return octal || "0";
}

console.log(decimalToOctalManual(64)); // "100"
```

◆ 44. Convert Decimal to Hexadecimal

✓ Built-in:

```
function decimalToHexBuiltIn(n: number): string {
    return n.toString(16).toUpperCase();
}

console.log(decimalToHexBuiltIn(255)); // "FF"
```

✓ Manual:

```
function decimalToHexManual(n: number): string {
    const hexChars = "0123456789ABCDEF";
    let hex = "";
    while (n > 0) {
        hex = hexChars[n % 16] + hex;
        n = Math.floor(n / 16);
    }
    return hex || "0";
}

console.log(decimalToHexManual(255)); // "FF"
```

◆ 45. Count Number of Set Bits

✓ Built-in:

```
function countSetBitsBuiltIn(n: number): number {
    return n.toString(2).split("1").length - 1;
}
```

```
console.log(countSetBitsBuiltIn(13)); // 3 (1101)
```

✓ Manual:

```
function countSetBitsManual(n: number): number {
  let count = 0;
  while (n > 0) {
    count += n & 1;
    n >>= 1;
  }
  return count;
}
```

```
console.log(countSetBitsManual(13)); // 3
```

◆ 46. Reverse Bits

✓ Built-in (using string manipulation):

```
function reverseBitsBuiltIn(n: number): number {
  return parseInt(n.toString(2).split('').reverse().join(''), 2);
}
```

```
console.log(reverseBitsBuiltIn(13)); // 11 (1101 → 1011)
```

✓ Manual:

```
function reverseBitsManual(n: number): number {
  let result = 0;
  while (n > 0) {
    result = (result << 1) | (n & 1);
    n >>= 1;
  }
  return result;
}
```

```
console.log(reverseBitsManual(13)); // 11
```

◆ 47. Check if a Number is Power of 2

✓ Built-in:

```
function isPowerOf2BuiltIn(n: number): boolean {
  return n > 0 && (n & (n - 1)) === 0;
}
```

```
console.log(isPowerOf2BuiltIn(16)); // true
```

✓ Manual:

```
function isPowerOf2Manual(n: number): boolean {
  if (n <= 0) return false;
  while (n > 1) {
    if (n % 2 !== 0) return false;
    n = Math.floor(n / 2);
  }
  return true;
}

console.log(isPowerOf2Manual(16)); // true
```

◆ 48. Square Root (without `Math.sqrt`)

✓ Built-in:

```
function sqrtBuiltIn(n: number): number {
  return Math.sqrt(n);
}

console.log(sqrtBuiltIn(25)); // 5
```

✓ Manual (binary search):

```
function sqrtManual(n: number): number {
  if (n < 2) return n;
  let low = 1, high = n, ans = 1;
  while (low <= high) {
    let mid = Math.floor((low + high) / 2);
    if (mid * mid <= n) {
      ans = mid;
      low = mid + 1;
    } else {
      high = mid - 1;
    }
  }
  return ans;
}

console.log(sqrtManual(25)); // 5
```

◆ 49. Fast Exponentiation (a^b)

✓ Built-in:

```
function powerBuiltIn(a: number, b: number): number {
  return Math.pow(a, b);
}

console.log(powerBuiltIn(2, 10)); // 1024
```

✓ Manual (fast exponentiation):

```
function powerManual(a: number, b: number): number {
  let result = 1;
  while (b > 0) {
    if (b % 2 === 1) result *= a;
    a *= a;
    b = Math.floor(b / 2);
  }
  return result;
}

console.log(powerManual(2, 10)); // 1024
```

◆ 50. Logarithm base 2 (without `Math.log2`)

✓ Built-in:

```
function log2BuiltIn(n: number): number {
  return Math.log2(n);
}

console.log(log2BuiltIn(16)); // 4
```

✓ Manual:

```
function log2Manual(n: number): number {
  let count = 0;
  while (n > 1) {
    n = Math.floor(n / 2);
    count++;
  }
  return count;
}

console.log(log2Manual(16)); // 4
```

?

◆ 51. Find the Greatest of N Numbers

✓ Built-in:

```
function greatestNumberBuiltIn(arr: number[]): number {  
    return Math.max(...arr);  
}  
  
console.log(greatestNumberBuiltIn([10, 25, 30, 5])); // 30
```

✓ Manual:

```
function greatestNumberManual(arr: number[]): number {  
    let max = arr[0];  
    for (let num of arr) {  
        if (num > max) max = num;  
    }  
    return max;  
}  
  
console.log(greatestNumberManual([10, 25, 30, 5])); // 30
```

◆ 52. Check if a Number is a Palindrome (without string conversion)

✓ Manual only:

```
function isPalindromeNumber(n: number): boolean {  
    let original = n, reversed = 0;  
    while (n > 0) {  
        reversed = reversed * 10 + (n % 10);  
        n = Math.floor(n / 10);  
    }  
    return original === reversed;  
}  
  
console.log(isPalindromeNumber(121)); // true
```

◆ 53. Find Sum of First and Last Digit

✓ Manual:

```
function sumFirstLastDigit(n: number): number {  
    let last = n % 10;  
    let first = n;  
    while (first >= 10) {  
        first = Math.floor(first / 10);  
    }  
}
```

```
    return first + last;
}

console.log(sumFirstLastDigit(1234)); // 5
```

◆ 54. Swap First and Last Digit

✓ Manual:

```
function swapFirstLastDigit(n: number): number {
    let str = n.toString().split("");
    let temp = str[0];
    str[0] = str[str.length - 1];
    str[str.length - 1] = temp;
    return parseInt(str.join(""));
}

console.log(swapFirstLastDigit(1234)); // 4231
```

Without string conversion:

```
function swapFirstLastDigitManual(n: number): number {
    let digits: number[] = [];
    let temp = n;
    while (temp > 0) {
        digits.unshift(temp % 10);
        temp = Math.floor(temp / 10);
    }
    [digits[0], digits[digits.length - 1]] = [digits[digits.length - 1],
    digits[0]];
    return parseInt(digits.join(""));
}

console.log(swapFirstLastDigitManual(1234)); // 4231
```

◆ 55. Find Product of Digits

✓ Manual:

```
function productOfDigits(n: number): number {
    let product = 1;
    while (n > 0) {
        product *= n % 10;
        n = Math.floor(n / 10);
    }
    return product;
}

console.log(productOfDigits(1234)); // 24
```

◆ 56. Find Digital Root of a Number

✓ Built-in (math trick):

```
function digitalRootBuiltIn(n: number): number {
  return 1 + ((n - 1) % 9);
}

console.log(digitalRootBuiltIn(9875)); // 2
```

✓ Manual:

```
function digitalRootManual(n: number): number {
  while (n >= 10) {
    let sum = 0;
    while (n > 0) {
      sum += n % 10;
      n = Math.floor(n / 10);
    }
    n = sum;
  }
  return n;
}

console.log(digitalRootManual(9875)); // 2
```

◆ 57. Count Trailing Zeros in Factorial of N

✓ Manual:

```
function trailingZerosInFactorial(n: number): number {
  let count = 0;
  for (let i = 5; i <= n; i *= 5) {
    count += Math.floor(n / i);
  }
  return count;
}

console.log(trailingZerosInFactorial(100)); // 24
```

◆ 58. Compute nCr (Combinations)

✓ Manual:

```
function nCr(n: number, r: number): number {
  function factorial(x: number): number {
    let fact = 1;
```

```
        for (let i = 1; i <= x; i++) fact *= i;
        return fact;
    }
    return factorial(n) / (factorial(r) * factorial(n - r));
}

console.log(nCr(5, 2)); // 10
```

◆ 59. Compute nPr (Permutations)

✓ Manual:

```
function nPr(n: number, r: number): number {
    function factorial(x: number): number {
        let fact = 1;
        for (let i = 1; i <= x; i++) fact *= i;
        return fact;
    }
    return factorial(n) / factorial(n - r);
}

console.log(nPr(5, 2)); // 20
```

◆ 60. Euclidean Algorithm for GCD

✓ Recursive:

```
function gcdEuclidean(a: number, b: number): number {
    return b === 0 ? a : gcdEuclidean(b, a % b);
}

console.log(gcdEuclidean(48, 18)); // 6
```

✓ Iterative:

```
function gcdEuclideanIterative(a: number, b: number): number {
    while (b !== 0) {
        let temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

console.log(gcdEuclideanIterative(48, 18)); // 6
```

◆ 61. Find All Armstrong Numbers in a Range

✓ Manual:

```
function isArmstrong(num: number): boolean {
  const digits = num.toString().split('').map(Number);
  const power = digits.length;
  const sum = digits.reduce((acc, d) => acc + Math.pow(d, power), 0);
  return sum === num;
}

function armstrongInRange(start: number, end: number): number[] {
  let result: number[] = [];
  for (let i = start; i <= end; i++) {
    if (isArmstrong(i)) result.push(i);
  }
  return result;
}

console.log(armstrongInRange(1, 1000)); // [1,153,370,371,407]
```

◆ 62. Find All Prime Numbers in a Range (Sieve of Eratosthenes)

✓ Manual:

```
function sieveOfEratosthenes(n: number): number[] {
  const primes: boolean[] = Array(n + 1).fill(true);
  primes[0] = primes[1] = false;

  for (let i = 2; i * i <= n; i++) {
    if (primes[i]) {
      for (let j = i * i; j <= n; j += i) {
        primes[j] = false;
      }
    }
  }

  return primes.map((isPrime, idx) => (isPrime ? idx : -1)).filter(x => x !== -1);
}

console.log(sieveOfEratosthenes(30)); // [2,3,5,7,11,13,17,19,23,29]
```

◆ 63. Count Primes Less Than N

```
function countPrimes(n: number): number {
  if (n <= 2) return 0;
  const primes: boolean[] = Array(n).fill(true);
  primes[0] = primes[1] = false;

  for (let i = 2; i * i < n; i++) {
    if (primes[i]) {
      for (let j = i * i; j < n; j += i) {
        primes[j] = false;
      }
    }
  }

  return primes.filter(Boolean).length;
}

console.log(countPrimes(20)); // 8
```

◆ 64. Check if a Number is Power of Another Number

```
function isPowerOfAnother(a: number, b: number): boolean {
  if (a === 1) return b === 1;
  let power = 1;
  while (power < b) {
    power *= a;
  }
  return power === b;
}

console.log(isPowerOfAnother(3, 81)); // true
```

◆ 65. Find the Sum of Divisors of a Number

✓ Manual:

```
function sumOfDivisors(n: number): number {
  let sum = 0;
  for (let i = 1; i <= Math.sqrt(n); i++) {
    if (n % i === 0) {
      sum += i;
      if (i !== n / i) sum += n / i;
    }
  }
  return sum;
}
```

```
}  
  
console.log(sumOfDivisors(12)); // 28 (1+2+3+4+6+12)
```

◆ 66. Count Total Divisors of a Number

```
function countDivisors(n: number): number {  
    let count = 0;  
    for (let i = 1; i * i <= n; i++) {  
        if (n % i === 0) {  
            count += (i * i === n) ? 1 : 2;  
        }  
    }  
    return count;  
}  
  
console.log(countDivisors(36)); // 9
```

◆ 67. Find the Highly Composite Number (smallest n with most divisors $\leq$ limit)

```
function highlyComposite(limit: number): number {  
    let maxDivisors = 0, result = 1;  
    for (let i = 1; i <= limit; i++) {  
        let divs = countDivisors(i);  
        if (divs > maxDivisors) {  
            maxDivisors = divs;  
            result = i;  
        }  
    }  
    return result;  
}  
  
console.log(highlyComposite(50)); // 48
```

◆ 68. Modular Exponentiation (Fast Power with mod)

```
function modExp(base: number, exp: number, mod: number): number {  
    let result = 1;  
    base = base % mod;  
    while (exp > 0) {  
        if (exp % 2 === 1) result = (result * base) % mod;  
        base = (base * base) % mod;  
        exp = Math.floor(exp / 2);  
    }  
    return result;  
}
```



```
console.log(modExp(2, 10, 1000)); // 24
```

◆ 69. Fast GCD using Binary GCD Algorithm (Stein's Algorithm)

```
function binaryGCD(a: number, b: number): number {
  if (a === 0) return b;
  if (b === 0) return a;

  let shift = 0;
  while (((a | b) & 1) === 0) {
    a >>= 1;
    b >>= 1;
    shift++;
  }

  while ((a & 1) === 0) a >>= 1;

  while (b !== 0) {
    while ((b & 1) === 0) b >>= 1;
    if (a > b) [a, b] = [b, a];
    b -= a;
  }

  return a << shift;
}

console.log(binaryGCD(48, 18)); // 6
```

◆ 70. Check if a Number is a Palindrome in All Bases up to K

```
function isPalindromeBase(num: number, base: number): boolean {
  const digits: number[] = [];
  let n = num;
  while (n > 0) {
    digits.push(n % base);
    n = Math.floor(n / base);
  }
  return digits.join("") === digits.reverse().join("");
}

function palindromeInAllBases(num: number, k: number): boolean {
  for (let base = 2; base <= k; base++) {
    if (!isPalindromeBase(num, base)) return false;
  }
  return true;
}

console.log(palindromeInAllBases(585, 10)); // false
```

◆ Miscellaneous Number Problems (71–80)

71. Convert number to words (e.g., 123 → "One Hundred Twenty Three")

```
function numberToWords(n: number): string {
  if (n === 0) return "Zero";

  const below20 = ["", "One", "Two", "Three", "Four", "Five", "Six",
    "Seven",
    "Eight", "Nine", "Ten", "Eleven", "Twelve", "Thirteen", "Fourteen",
    "Fifteen", "Sixteen", "Seventeen", "Eighteen", "Nineteen"];
  const tens = ["", "", "Twenty", "Thirty", "Forty", "Fifty",
    "Sixty", "Seventy", "Eighty", "Ninety"];

  function helper(num: number): string {
    if (num === 0) return "";
    else if (num < 20) return below20[num] + " ";
    else if (num < 100) return tens[Math.floor(num / 10)] + " " +
      helper(num % 10);
    else return below20[Math.floor(num / 100)] + " Hundred " + helper(num %
      100);
  }

  let result = "";
  const units = ["", "Thousand", "Million", "Billion"];
  let i = 0;
  while (n > 0) {
    if (n % 1000 !== 0) {
      result = helper(n % 1000) + units[i] + " " + result;
    }
    n = Math.floor(n / 1000);
    i++;
  }
  return result.trim();
}
```

72. Next greater number with same digits

```
function nextGreaterNumber(n: number): number {
  let digits = n.toString().split("").map(Number);
  let i = digits.length - 2;

  // Step 1: Find pivot
  while (i >= 0 && digits[i] >= digits[i + 1]) i--;
  if (i < 0) return -1;

  // Step 2: Find next greater digit
  let j = digits.length - 1;
  while (digits[j] <= digits[i]) j--;

  // Step 3: Swap
  [digits[i], digits[j]] = [digits[j], digits[i]];

  // Step 4: Reverse right part
  let left = i + 1, right = digits.length - 1;
  while (left < right) {
    [digits[left], digits[right]] = [digits[right], digits[left]];
    left++;
    right--;
  }
  return parseInt(digits.join(""));
}
```

```

while (left < right) {
  [digits[left], digits[right]] = [digits[right], digits[left]];
  left++; right--;
}
return parseInt(digits.join(""));
}

```

73. Largest number formed by digits

```

function largestFromDigits(n: number): number {
  return parseInt(n.toString().split("").sort((a, b) => Number(b) - Number(a)).join(""));
}

```

74. Check if digits are increasing/decreasing

```

function checkOrder(n: number): string {
  const digits = n.toString().split("").map(Number);
  let inc = true, dec = true;

  for (let i = 1; i < digits.length; i++) {
    if (digits[i] > digits[i - 1]) dec = false;
    if (digits[i] < digits[i - 1]) inc = false;
  }

  if (inc) return "Increasing";
  if (dec) return "Decreasing";
  return "Neither";
}

```

75. Generate all possible numbers from digits

```

function generateNumbers(digits: number[]): number[] {
  const result: number[] = [];
  const used: boolean[] = new Array(digits.length).fill(false);

  function backtrack(curr: number[]) {
    if (curr.length > 0) {
      result.push(parseInt(curr.join("")));
    }
    for (let i = 0; i < digits.length; i++) {
      if (!used[i]) {
        used[i] = true;
        curr.push(digits[i]);
        backtrack(curr);
        curr.pop();
        used[i] = false;
      }
    }
  }
  backtrack([]);
  return result;
}

```

76. Count numbers with unique digits up to n

```

function countUniqueDigits(n: number): number {
  function hasUnique(num: number): boolean {
    const seen = new Set<string>();
    for (let ch of num.toString()) {

```

```

        if (seen.has(ch)) return false;
        seen.add(ch);
    }
    return true;
}

let count = 0;
for (let i = 0; i <= n; i++) {
    if (hasUnique(i)) count++;
}
return count;
}

```

77. Digital complement of a number

```

function digitalComplement(n: number): number {
    return parseInt(n.toString().split('').map(d => (9 -
parseInt(d)).toString()).join(''));
}

```

78. Strong Number (sum of factorial of digits = number)

```

function isStrongNumber(n: number): boolean {
    function factorial(x: number): number {
        let fact = 1;
        for (let i = 2; i <= x; i++) fact *= i;
        return fact;
    }

    let sum = 0, temp = n;
    while (temp > 0) {
        sum += factorial(temp % 10);
        temp = Math.floor(temp / 10);
    }
    return sum === n;
}

```

79. Happy Number

```

function isHappyNumber(n: number): boolean {
    function sumOfSquares(num: number): number {
        let sum = 0;
        while (num > 0) {
            let d = num % 10;
            sum += d * d;
            num = Math.floor(num / 10);
        }
        return sum;
    }

    const seen = new Set<number>();
    while (n !== 1 && !seen.has(n)) {
        seen.add(n);
        n = sumOfSquares(n);
    }
    return n === 1;
}

```

80. Kaprekar Number

```
function isKaprekarNumber(n: number): boolean {  
    let sq = (n * n).toString();  
    let d = n.toString().length;  
    let right = parseInt(sq.slice(-d)) || 0;  
    let left = parseInt(sq.slice(0, sq.length - d)) || 0;  
    return left + right === n;  
}
```

sheet (like we did for Arrays & Strings), so you can copy it into a `.docx` or `.pdf` easily?